Incremental Placement Optimization Beyond Detailed Placement: Simultaneous Gate Sizing, Buffering, and Cell Relocation

Yi-Chen Lu, Rongjian Liang, Wen-Hao Liu, and Haoxing (Mark) Ren

NVIDIA

0. Revision History

2025-09-03: update runtime scoring

2025-07-30: minor update on typos

2025-07-23: Update on submission and output format

2025-06-21: Environment Setup Update; Scoring Explanation.

2025-05-06: Environment Setup

2025-05-05: Improve scoring explanation

1. Introduction:

Incremental placement optimization is a critical stage in modern Physical Design (PD), where the goal is to refine key Power, Performance, and Area (PPA) metrics derived from an initial placement. However, conventional incremental placement flows are inherently iterative, relying on repeated rounds of physical optimization, including gate sizing, buffering, and cell relocation, each followed by Static Timing Analysis (STA) to verify PPA improvements. This process is not only time-consuming but also sub-optimal by nature, as each new transformation must account for all prior modifications and constraints. For example, an optimization that resolves one critical path can easily worsen another path or violate legality constraints. Therefore, there is a pressing need for more holistic methods that optimize PPA in a global and systematic manner.

This contest specifically targets these challenges by encouraging contestants to go beyond ad-hoc, localized tweaks. Rather than restricting transformations to small, isolated physical adjustments, contestants are invited to adopt a unified approach encompassing gate sizing, buffer insertion, and cell relocation. By integrating these techniques, the competition seeks to unlock significantly higher design quality than what can be achieved through conventional incremental flows, enabling better trade-offs among timing, power, and wirelength.

Importantly, the final placement solutions must balance these global improvements with strict adherence to placement legality and design rules.

To handle the scale of real-world designs and to perform global PPA optimization at scale, contestants are strongly encouraged (*but not required*) to explore modern Machine Learning (ML) infrastructure as PyTorch [1] that offers gradient-based optimization through GPU-accelerated backpropagation. Recently, such techniques have shown great promise in PD for handling large scale optimization problems effectively [2] [3] [4] [5]. While the use of GPUs or ML is optional, contestants should be aware that creative use of these technologies could provide a significant advantage in both solution quality and runtime. The emphasis, however, remains on achieving the best PPA improvement possible, by whatever efficient means, beyond what traditional tools can do.

2. Contest Objectives:

The objective of this contest is to enhance the PPA of an initial seed placement by a coordination of the following methods:

1. **Relocating Cells**: Implement global cell movement (both standard cells and macros) that is both effective and minimally disruptive (i.e., displacement).
2. **Gate Sizing**: Select alternative library cell variants (e.g., with different drive strengths, areas, or VT flavors) to achieve optimal timing-power balance.
3. **Buffer Insertion**: Introduce buffers (including inverter pairs) at where necessary to drive down delays on critical or high fanout nets with minimal area overhead.

Note that **all modifications to the seed placement must result in legal design**. That is, cells must be on valid sites, there should be no overlaps, and IO ports must remain unchanged. There is a netlist functionality check as well.

3. Problem Formulation

3.1 Input Formats

In this contest, you will receive all necessary files to tackle the incremental placement optimization challenge. The design data is provided using standard EDA file formats, as described below:

- **Verilog Netlist (.v)**: a gate-level netlist that defines the logical connectivity
- **DEF File (.def)**: a complete Design Exchange Format of the initial placement
- **Bookshelf Files [6]**:

- **.nodes:** a list of design instances with their dimensions
- **.nets:** connectivity information between pins
- **.pl**: the cell coordinates of the seed placement
- **.scl**: defines the floorplan (e.g., core region and placement grid)

- **ASAP7 Library Files**: this contest utilizes ASAP7 libraries, which provide:
  - **.lib**: the timing and power library with lookup tables for slew/delay/power
  - **.lef**: the technology and cell LEF files describe physical dimensions

An example design set will be provided to illustrate the file structure and serve as a reference for the expected input. This example will include a complete Bookshelf-format design along with the corresponding ASAP7 library files.

3.2 Output Formats

Contestants are required to submit an updated placement file that adheres to the DEF format. The primary outputs are:

1. The updated **.def file** reflecting your final solution after applying cell relocations, sizing , and buffering.
2. An **ECO changelist** detailing the cell sizing and buffering transformations in order
   a. Sizing command:
      i. *size_cell <cellName> <libCellName>*

   Explanation:
   1. <cellName> must exist in the provided gate-level verilog netlist.
   2. <libCellName> must exist in the provided library (.lib and .lef).
   3. The new <libCellName> must have the same function as the original.

   b. Buffering commands:
      i. *insert_buffer <{one or more load pin names}> <library buffer name> <new buffer cell name> <new buffered net name>*
      ii. *insert_buffer -inverter_pairs {one or more load pin names} <library inverter name> {<new inverter cell names>} {<new net names>}*

   Explanation:
   1. We follow Synopsys PrimeTime's format [7] for buffering.
   2. Buffering in this contest includes inserting buffers and inverter pairs.
   3. The new buffer/inverter names must match in the updated **.pl** file.
   4. For inverter pair insertion, repeating inverters will be inserted to netlist.

Contestants must specify the name of each of them along with the newly created net names.

Contestants must submit a **.changelist** file (can be empty) indicating what are the sizing and buffering transforms being done to the original netlist. Note that the commands within the changelist are processed from top to bottom, meaning later transformations override any earlier ones. Legality will be checked. Any invalid eco command will be skipped.

The returned outputs must meet the following criteria:

- **Legal Placement:** all cells must be positioned on valid placement sites as defined in the provided .scl and .lef files with no overlaps.
- **Fixed IO Ports:** the positions of IO ports and any fixed macros/cells must remain unchanged.
- **Library Consistency:** the output files should be fully consistent with the input netlist and library constraints as described above.

3.3 Scoring and Evaluation Metrics:

Submissions will be assessed based on a composite score that captures improvements in PPA while penalizing excessive cell movement and inefficient runtimes. The overall score S is computed using the following components:

- PPA Improvement (P):
  - Timing Improvement (TNS_{norm}):
    - $TNS_{norm} = \frac{TNS_{solution} - TNS_{seed}}{|TNS_{seed}|}$
  - Power Reduction ($Power_{norm}$):
    - $Power_{norm} = \frac{Power_{solution} - Power_{seed}}{Power_{seed}}$
  - Wirelength Reduction (WL_{norm}):
    - $WL_{norm} = \frac{WL_{solution} - WL_{seed}}{WL_{seed}}$

  The overall PPA improvement score P is then calculated as:

$$P = \alpha * TNS_{norm} + \beta * Power_{norm} + \gamma * WL_{norm},$$

  where the weighting factors α, β, γ will be subject to each testcase.

  PPA metrics will be verified by OpenROAD [8]. Evaluation script will be provided.

  Seed refers to the average score of all legal solutions from all participating teams.

- Averaged Displacement Penalty (D):

To discourage overly disruptive changes, the average Manhattan displacement per cell (measured in valid site units) is computed. A higher average displacement leads to a greater penalty:

$$D = \boldsymbol{Average}\ Manhattan\ Displacement\ per\ Cell$$

D will be normalized against *seed* as above P.

- Runtime Efficiency (R):

  The total runtime of your algorithm is normalized over a predefined reference per testcase. There will be a maximum runtime limit subject to each testcase. Details will be posted soon.

3.4 Final Score Calculation (S) Per Design:

The final composite score S is defined as:

$$S = 1000 \times P - 50 \times D - 30 \times R$$

Note that all P, D, and R can be interpreted as "percentage of improvement" over seed metrics. As previously mentioned, one of the contest's primary objectives is to encourage GPU-accelerated techniques. Consequently, runtime performance will be a critical factor in the final evaluation. Submissions will be ranked in descending order of S. This scoring framework is designed to recognize strategies that deliver substantial PPA improvements while minimizing cell displacement and ensuring runtime efficiency.

4. Evaluation Environment

All submissions will be run inside a Docker image built from the provided Dockerfile on an Ubuntu 20.04 host with:

- CPU: 8 × Intel Xeon vCPUs
- RAM: 32 GB
- GPU: 1 × NVIDIA A100 (40 GB HBM2, compute capability 8.0)
- CUDA Runtime: 11.8 (as per nvidia/cuda:11.8.0-runtime-ubuntu20.04)
- Drivers: NVIDIA Driver 525.x
- Conda: Mambaforge-installed Python 3.9 environment (via lagrange_env.yaml)

Contestants will be provided with a Dockerfile alongside a yaml file defining the exact environment used for evaluation. All tools and code must be executable within this Docker environment.

5. Submission and Output Format

All contestants are required to submit a compressed file named solution.tar.gz. Upon extraction using the command: "*tar -zxvf solution.tar.gz*" a directory named solution/ must be created with the following structure:

solution/ ← this is extracted after "tar -zxvf solution.tar.gz"

├── setup_environment.sh → will be executed first

├── run.sh → will be executed second

├── <your folders / files / binaries etc.>

└── testcases/ ← (**provided by the host**)

  └── <design_name>/

  └── <ASAP7>

1. A script named **setup_environment.sh** located directly under the solution/ folder. This script should install all necessary dependencies.
2. A script named **run.sh**, also under solution/, which will be used to run your solution. This script must assume that the folder testcases/ is located in the same directory as itself, and must accept exactly four arguments, in the following order:

   ./run.sh <design_name> <TNS_weight, α > <power_weight, β > <WL_weight, γ >

   Where:
   <design_name> denotes a folder under testcases/ (e.g., testcases/aes/)
   <WL_weight> is the weight for wirelength.
   <timing_weight> is the weight for timing.
   <power_weight> is the weight for power.

Your script is expected to generate two output files, both placed under the solution/ directory:

- <design_name>.sol.def
- <design_name>.sol.changelist

The *.sol.def file describes the final netlist after cell relocations, sizing, and buffering operations applied to the original design. The *.sol.changelist file should represent sizing

and buffering procedures that are taken. During evaluation, we will verify correctness by applying the changelist to the original DEF and checking whether the resulting DEF matches your submitted *.sol.def.

Please make sure your submission strictly follows this structure to avoid disqualification during automated evaluation.

6. Reference:

[1] Imambi, Sagar, Kolla Bhanu Prakash, and G. R. Kanagachidambaresan. "PyTorch." *Programming with TensorFlow: solution for edge computing applications* (2021): 87-104.

[2] Du, Yufan, et al. "Fusion of Global Placement and Gate Sizing with Differentiable Optimization." *ICCAD*. 2024.

[3] Guo, Zizheng, and Yibo Lin. "Differentiable-timing-driven global placement." *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 2022.

[4] Lin, Yibo, et al. "Dreamplace: Deep learning toolkit-enabled gpu acceleration for modern vlsi placement." *Proceedings of the 56th ACM/IEEE Annual Design Automation Conference 2019*.

[5] Lu, Yi-Chen, et al. "INSTA: An Ultra-Fast, Differentiable, Statistical Static Timing Analysis Engine for Industrial Physical Design Applications" *Proceedings of the 62th ACM/IEEE Annual Design Automation Conference 2025*.

[6] Bookshelf format reference: http://vlsicad.eecs.umich.edu/BK/ISPD06bench/BookshelfFormat.txt

[7] PrimeTime User Guide, Advanced Timing Analysis. V-2018.03, Synopsys Online Documentation

[8] Ajayi, Tutu, and David Blaauw. "OpenROAD: Toward a self-driving, open-source digital layout implementation tool chain." *Proceedings of Government Microcircuit Applications and Critical Technology Conference*. 2019.